

Endpoint Security Monitoring

Wazuh SIEM Agent Deployment, Vulnerability Detection & CIS Benchmark
Hardening on a Windows 11 Host

Prepared by Khizar UI Islam

CEH Certified | SOC Analyst (L1) Candidate

mrkhizar97@gmail.com | github.com/Khizar-malik97 | linkedin.com/in/khizarulislam79

Lab Environment

Wazuh single-node manager v4.9.x (192.168.44.136) monitoring a Windows 11 Pro 24H2 endpoint via the official Wazuh agent v4.14.5. Environment built and operated end-to-end as a hands-on SOC home-lab exercise.

Executive Summary

This report documents the end-to-end deployment of a Wazuh agent on a Windows 11 endpoint, integration with a Wazuh manager for centralized log collection and monitoring, and a full security assessment cycle covering vulnerability detection and CIS benchmark compliance. The exercise simulates a typical SOC analyst workflow: deploy visibility, identify risk, prioritize findings, remediate, and verify.

Key Outcomes

- **Endpoint onboarded:** Successfully deployed and enrolled a Wazuh agent on a Windows 11 Pro host, verified active connectivity to the manager.
- **Critical vulnerability discovered:** Wazuh's Vulnerability Detection module identified 17 vulnerabilities across installed software, including a CVSS 9.9 critical CVE (CVE-2025-55315) in ASP.NET Core / Kestrel.
- **Compliance gap identified:** A CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0 scan established a 26% compliance baseline against 482 individual checks.
- **Manual remediation performed:** Applied hardening across Password Policy, Account Lockout Policy, Audit Policy, account naming, and Interactive Logon settings.
- **Verified improvement: 26% → 27% compliance score** delta confirmed via rescan, with the three remediated password-policy checks (IDs 26000-26002) flipping from Failed to Passed.

Metric	Before	After
CIS Compliance Score	26%	27%
Checks Passed	124 / 482	128 / 482
Checks Failed	349 / 482	345 / 482
Critical Vulnerabilities	2	Remediation in progress

1. Wazuh Agent Deployment on Windows 11

The Wazuh manager dashboard's built-in “Deploy new agent” wizard was used to generate a pre-configured installation command for the Windows endpoint, pointing it at the manager's IP address (192.168.44.136).

1.1 Initial Manager State

Before deployment, the manager showed zero registered agents. The Overview dashboard also surfaces a 24-hour alert summary by severity (driven by the manager's own internal logs at this stage).

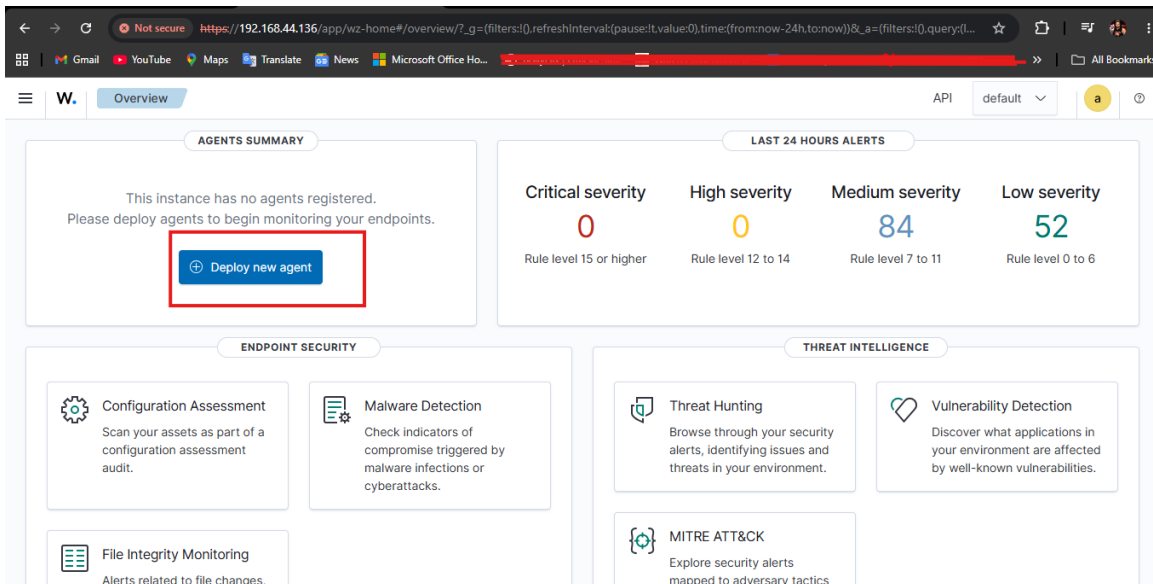


Figure 1 — Wazuh manager Overview dashboard prior to agent deployment, showing no registered agents.

1.2 Agent Package Selection

The Windows MSI package (32/64-bit) was selected as the install target, with the manager's IP address supplied as the server address and a descriptive agent name assigned for clarity in the agent inventory.

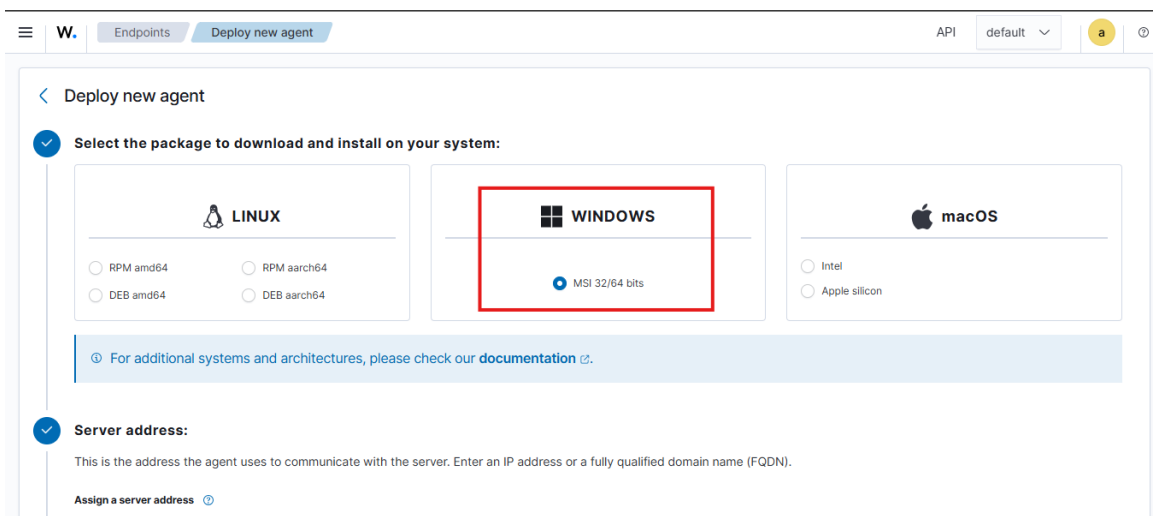


Figure 2 — Selecting the Windows MSI package in the Deploy New Agent wizard.

W. Endpoints Deploy new agent API default

Server address:
This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address [?](#)
192.168.44.136
☒ Remember server address

Optional settings:
By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name: [?](#)
Khizar-Malik-Windows11

[?](#) The agent name must be unique. It can't be changed once the agent has been enrolled. [?](#)

Figure 3 — Assigning the manager's server address (192.168.44.136) and a unique agent name (Khizar-Malik-Windows11).

1.3 Installation & Service Start

Wazuh generated a ready-to-run PowerShell one-liner that downloads the agent MSI and installs it silently with the manager address and agent name pre-populated as MSI parameters.

W. Endpoints Deploy new agent API default a

4 Run the following commands to download and install the agent:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi -OutFile $env:tmp\wazuh-agent; msixec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.44.136' WAZUH_AGENT_NAME='Khizar-Malik-Windows11'
```

Requirements

- You will need administrator privileges to perform this installation.
- PowerShell 3.0 or greater is required.

Keep in mind you need to run this command in a Windows PowerShell terminal.

5 Start the agent:

```
NET START Wazuh
```

Figure 4 — Auto-generated installation command and post-install NET START instruction.

The command was executed in an elevated (Administrator) PowerShell session, as required by the installer:

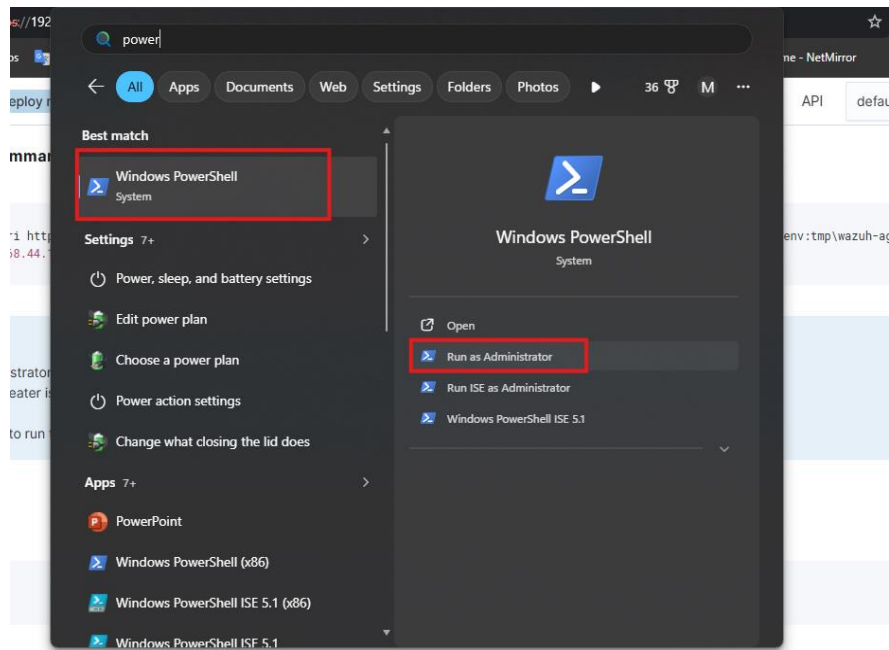


Figure 5 — Launching Windows PowerShell with Administrator privileges.

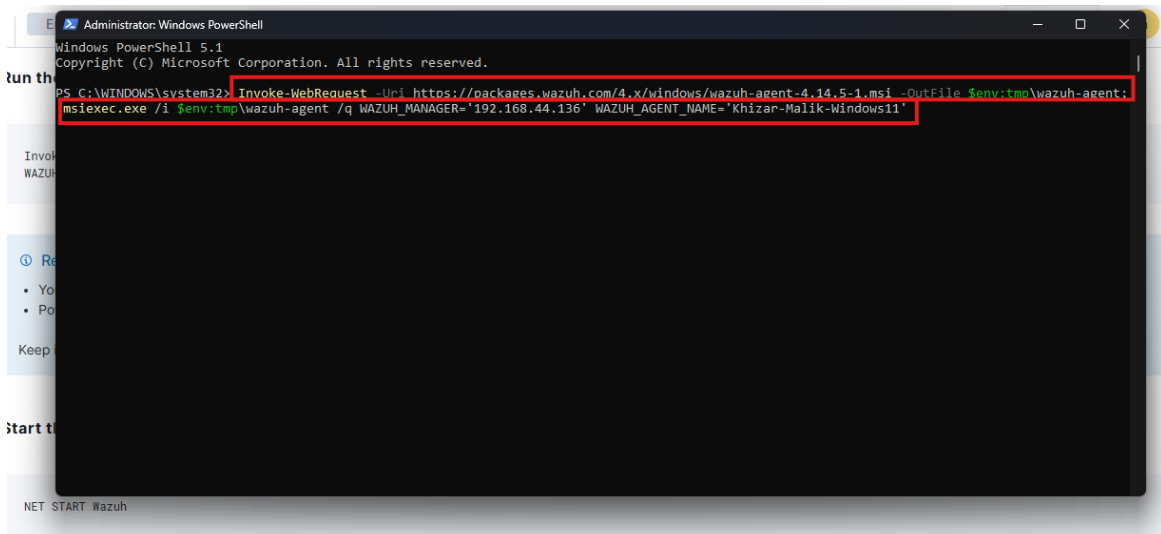


Figure 6 — Running the Wazuh agent download and silent-install command.

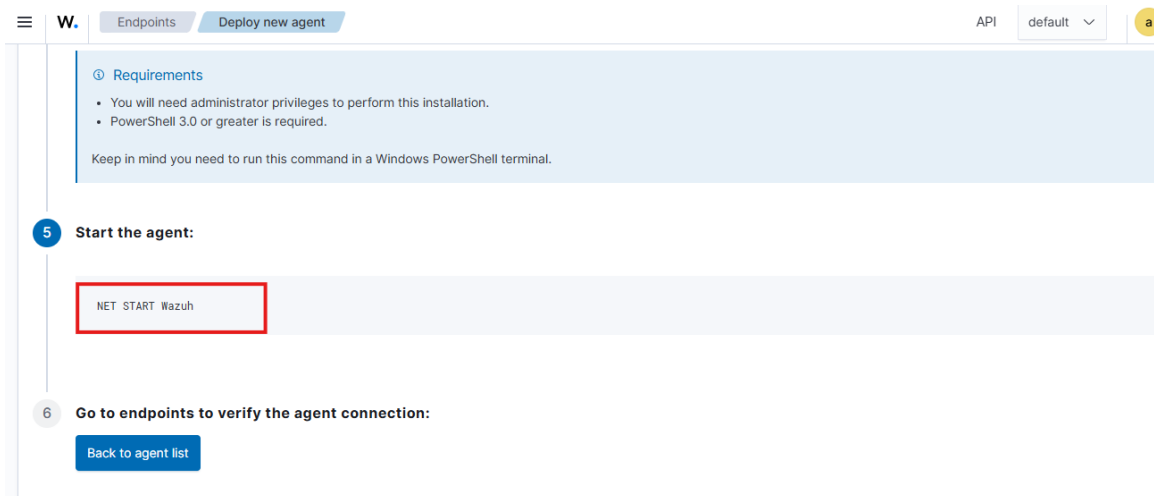


Figure 7 — Wazuh deployment wizard, Step 5: starting the agent service.

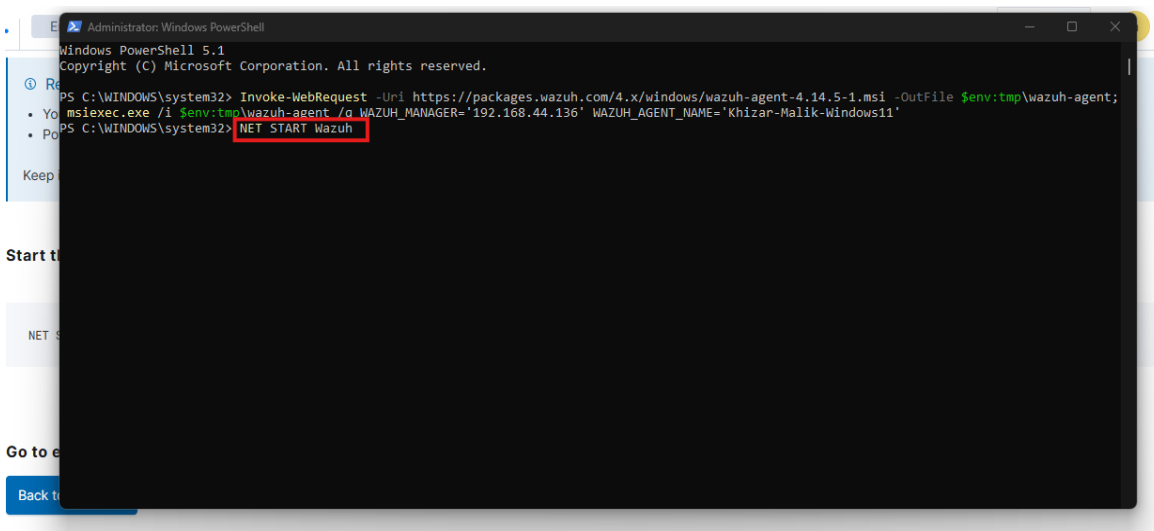


Figure 8 — Issuing NET START Wazuh to start the agent service.

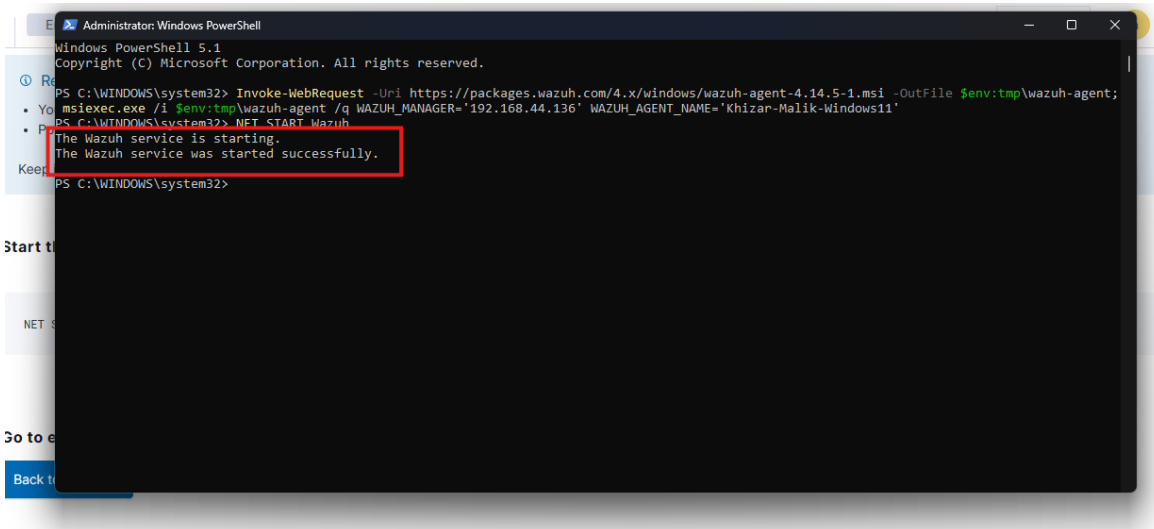


Figure 9 — Confirmation that the Wazuh service started successfully.

1.4 Local Installation Verification

The agent installation was independently verified on the endpoint via Windows Settings → Apps → Installed apps, confirming Wazuh Agent v4.14.5 was correctly installed.

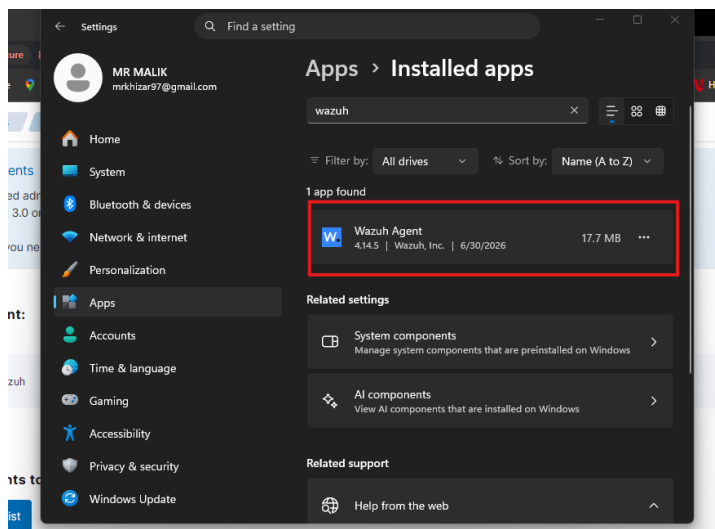


Figure 10 — Wazuh Agent v4.14.5 confirmed in Windows installed applications.

1.5 Manager-Side Enrollment Confirmation

Back on the Wazuh dashboard's Endpoints view, the agent appeared with an Active status, correctly identified as a Windows 11 Pro host (build 10.0.26100.8737) running agent version 4.14.5, grouped under the default policy group.

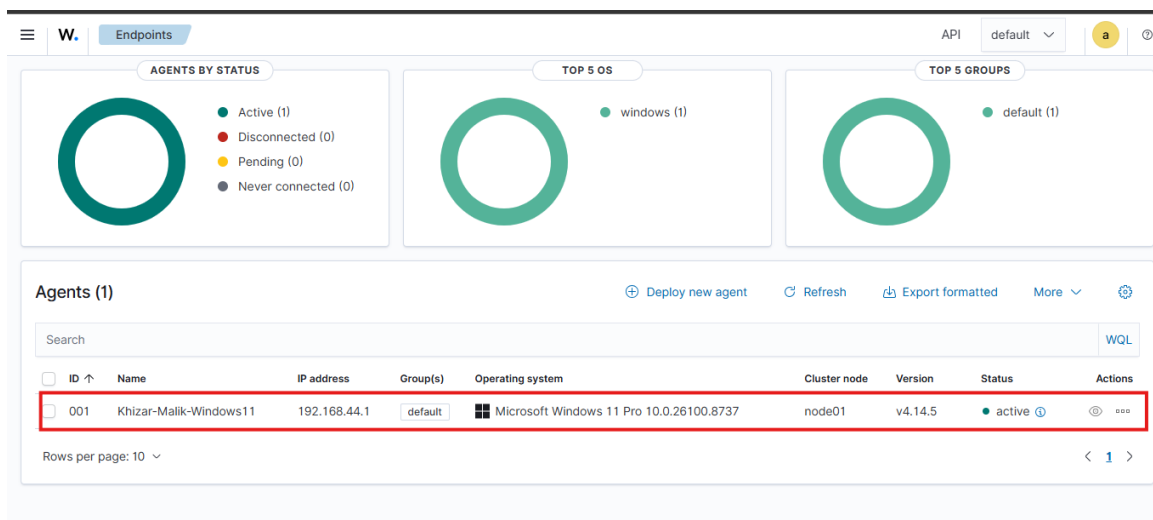


Figure 11 — Endpoint successfully enrolled and reporting Active status on the Wazuh manager.

2. Vulnerability Detection

With log and inventory collection active, Wazuh's Vulnerability Detection module automatically cross-referenced installed software against known CVE databases for the endpoint.

2.1 Findings Summary

Severity	Count
Critical	2
High	11
Medium	4
Low	0

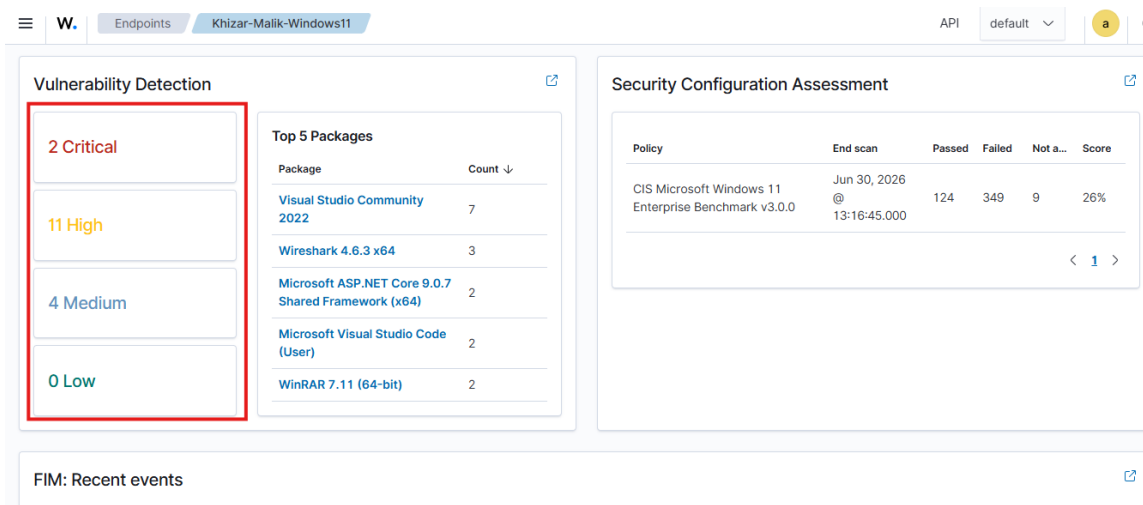


Figure 12 — Endpoint summary panel showing Vulnerability Detection results alongside CIS Configuration Assessment.

2.2 Critical Finding: CVE-2025-55315

The top critical finding, affecting the host's ASP.NET Core 9.0.7 Shared Framework installation, was investigated in detail.

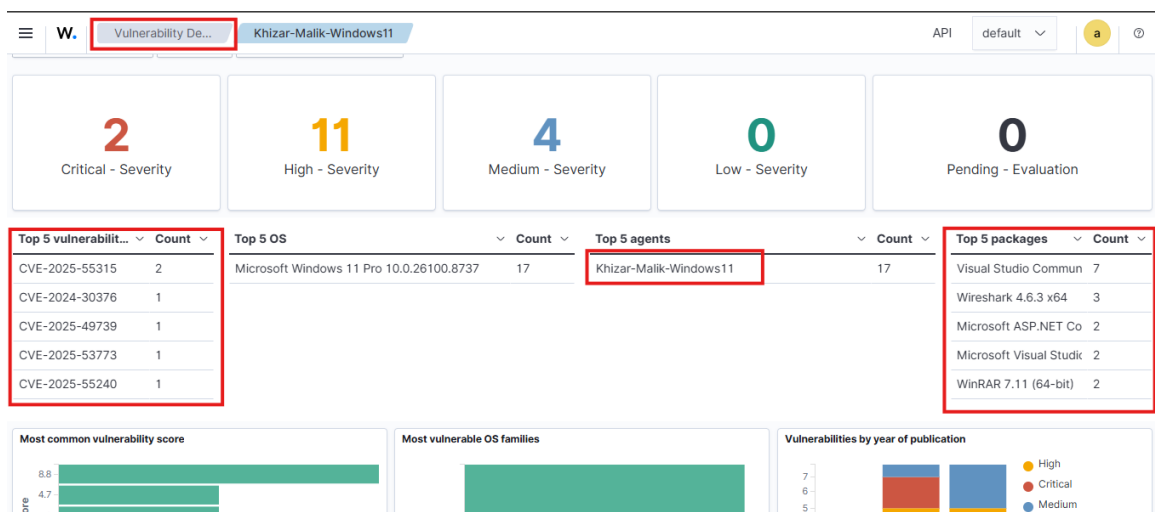


Figure 13 — Full Vulnerability Detection view: severity breakdown, top CVEs, and top affected packages.

CVE-2025-55315 — ASP.NET Core / Kestrel HTTP Request Smuggling (CVSS 9.9, Critical)

A critical security feature bypass in ASP.NET Core's Kestrel web server, caused by inconsistent interpretation of HTTP requests using chunked transfer encoding with malformed chunk extensions. This ambiguity allows an attacker to “smuggle” a hidden request inside another, potentially bypassing authentication and authorization checks. Disclosed by Microsoft on 14 October 2025 with the highest CVSS score ever assigned to an ASP.NET Core CVE. Affects ASP.NET Core 8.0.0-8.0.20, 9.0.0-9.0.9, and 10.0.0-rc2. Remediation: update the .NET SDK/runtime to the patched release (8.0.21+, 9.0.10+, or 10.0.0-rc2+) via Visual Studio or the .NET download site.

3. CIS Benchmark Compliance Assessment

Wazuh's Security Configuration Assessment (SCA) module ran a full scan against the CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0 — 482 individual policy checks covering account policies, audit settings, user rights, and security options.

3.1 Baseline Scan Results

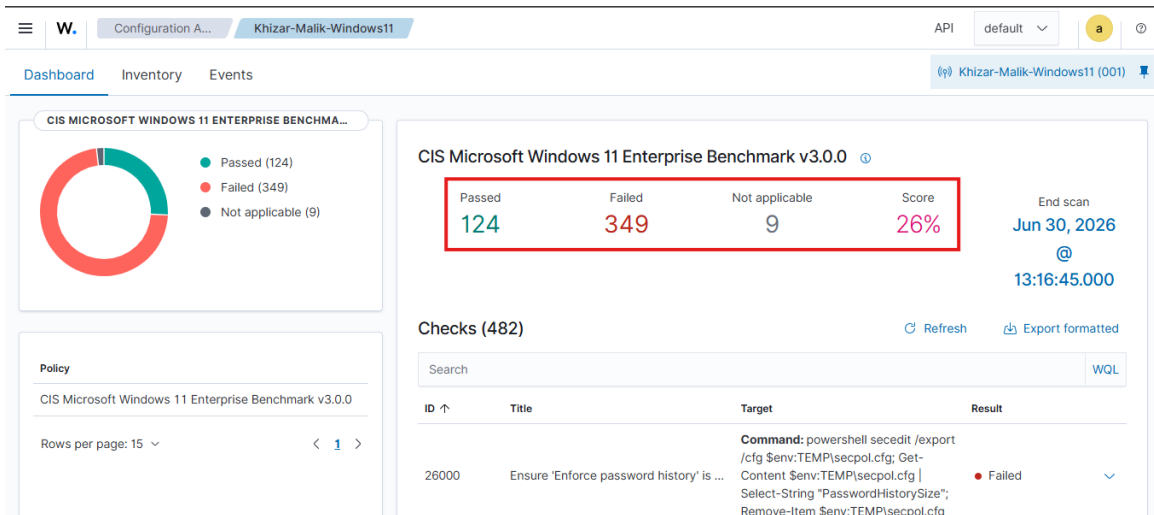


Figure 14 — CIS Benchmark baseline: 124 passed, 349 failed, 9 not applicable — 26% compliance score.

Each failed check includes the exact PowerShell/registry command Wazuh used to test it, a rationale, and a remediation path — turning the scan into an actionable checklist rather than a static report.

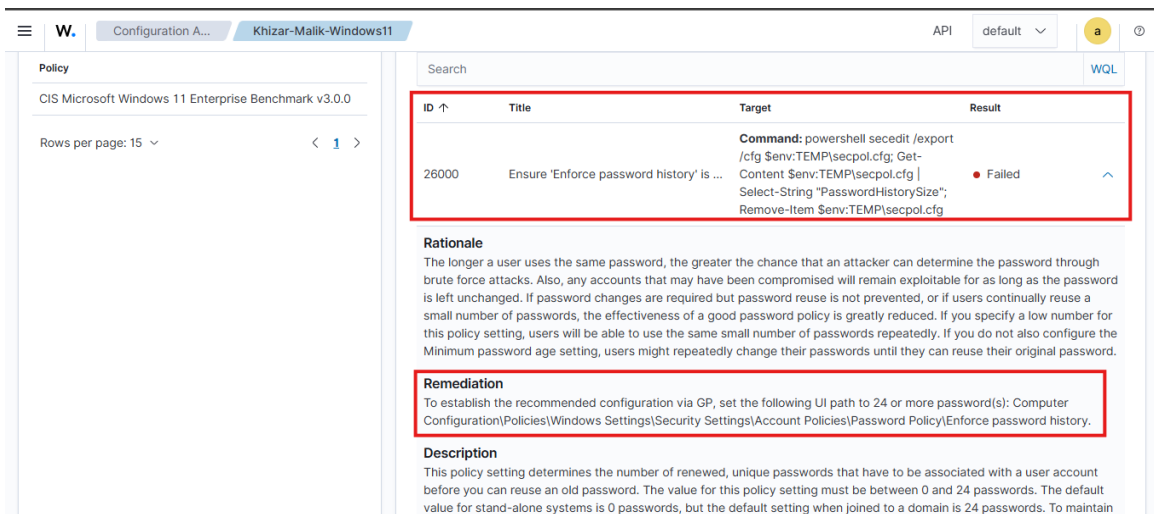


Figure 15 — Example failed check (ID 26000, 'Enforce password history') with rationale and remediation guidance.

3.2 Prioritized Remediation

Given the scale of the finding set (482 checks), remediation was scoped to the highest-value, lowest-risk categories first: Account Policies (Password Policy, Account Lockout Policy) and a subset of Local Policies → Security Options, applied directly via the Local Security Policy editor (secpol.msc).

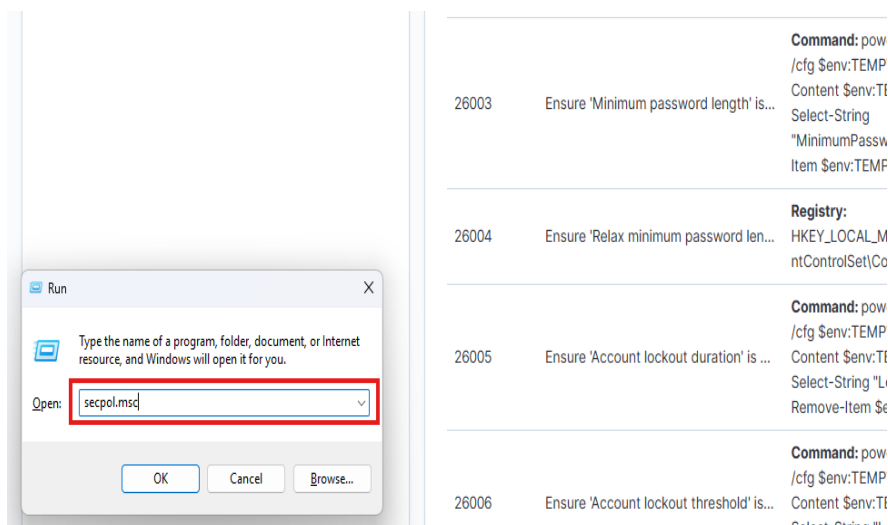


Figure 16 — Launching the Local Security Policy editor (secpol.msc) to apply remediations directly on the endpoint.

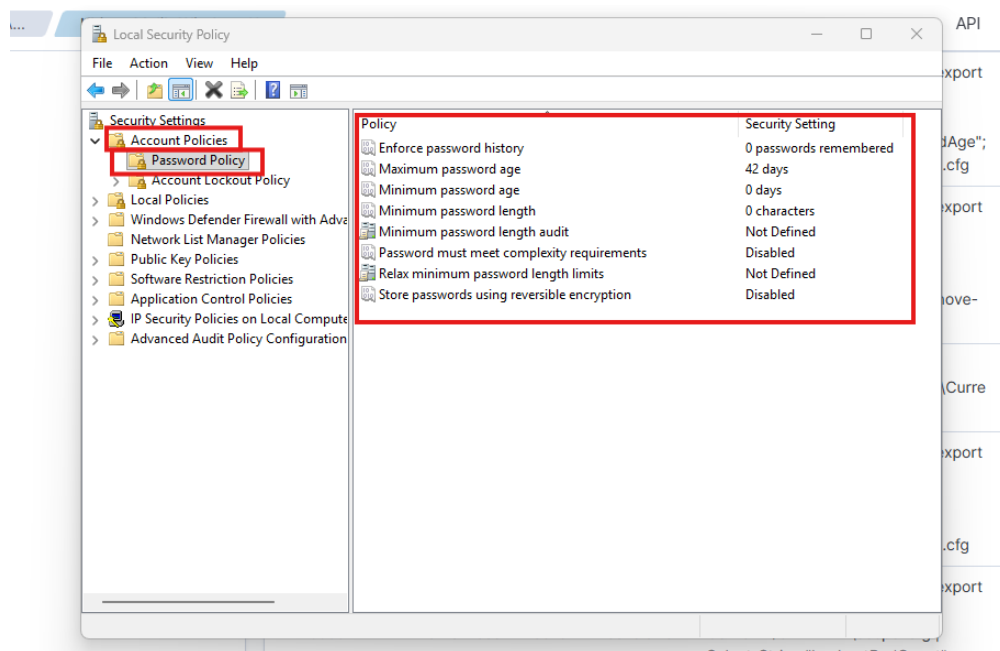


Figure 17 — Password Policy values prior to remediation — history, minimum age, and minimum length all non-compliant.

Check ID	Policy	Before	Remediated To
26000	Enforce password history	0 remembered	24 remembered
26002	Minimum password age	0 days	1 day
26003	Minimum password length	0 characters	14 characters
—	Account lockout threshold	10 attempts	5 attempts
—	Rename administrator account	Administrator	LocalAdmin01

—	Rename guest account	Guest	LocalGuest01
—	Force audit policy subcategory override	Not Defined	Enabled
—	Devices: allow undock without logon	Enabled	Disabled

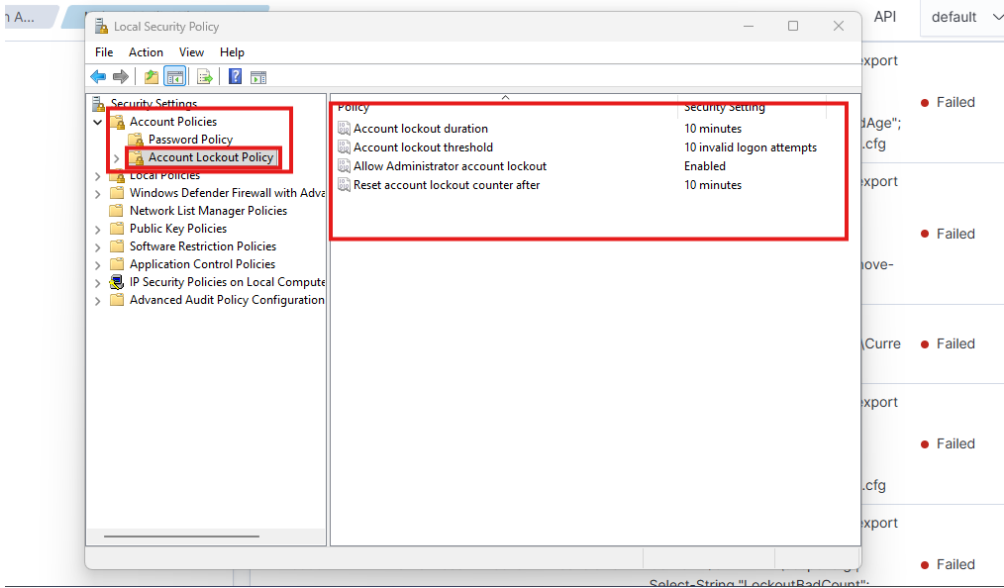


Figure 18 — Account Lockout Policy reviewed and tightened (lockout threshold reduced to 5 invalid attempts).

3.3 Post-Remediation Verification

After applying the changes, the Wazuh agent service was restarted to trigger a fresh SCA scan. The rescanned results confirmed all three targeted password-policy checks (26000, 26001, 26002) transitioned from Failed to Passed, and the overall score increased.

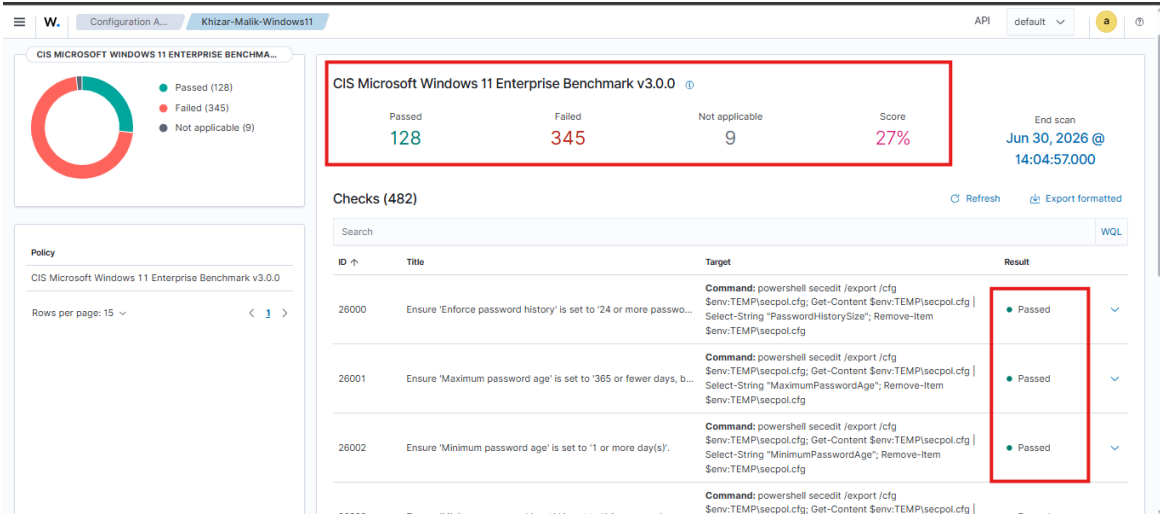


Figure 19 — Post-remediation rescan: compliance score improved from 26% to 27%, with targeted checks (26000-26002) confirmed Passed.

4. Conclusions & Next Steps

This exercise covered the full lifecycle of endpoint security monitoring with Wazuh: deployment, telemetry collection, vulnerability identification, compliance assessment, and remediation — the same workflow expected of a SOC L1 analyst when triaging and acting on findings.

4.1 Skills Demonstrated

- **SIEM agent lifecycle management:** deploying and enrolling endpoint agents via manager-generated installation packages.
- **Vulnerability triage:** interpreting CVSS scoring, understanding root cause (HTTP request smuggling via chunked transfer encoding), and identifying correct patch paths.
- **Security hardening:** reading and applying CIS benchmark remediation guidance using native Windows tooling (secpol.msc).
- **Verification discipline:** confirming fixes took effect via a second scan rather than assuming success.

4.2 Planned Follow-On Work

- install and integrate Sysmon (SwiftOnSecurity baseline config) for process, network, and registry-level telemetry.
- continue remediating Local Policies → Security Options and Audit Policy categories to drive compliance score further upward.
- update the vulnerable ASP.NET Core runtime and re-scan to confirm the Critical CVE count drops to zero.
- deploy Suricata IDS, build an Active Directory attack lab, and author MITRE ATT&CK-mapped detection rules and SOC playbooks as outlined in the broader home-lab roadmap.